Customer Churn Prediction Deep Learning project:

In [165]:

```
# Import required libraries:
import pandas as pd
import matplotlib.pyplot as plt
import numpy as np

#Ignore all warnings (if any upcoming):
import warnings
warnings.filterwarnings("ignore")

#Load the dataset:
df = pd.read_csv("customer_churn.csv")
```

In [166]:

```
# 5 samples of dataset:
df.sample(5)
```

Out[166]:

	customerID	gender	SeniorCitizen	Partner	Dependents	tenure	PhoneService	MultipleLines	InternetService	OnlineSe
2944	5311-IHLEI	Male	0	No	No	12	No	No phone service	DSL	
1214	9866-QEVEE	Male	0	No	No	19	Yes	Yes	Fiber optic	
4868	2568-OIADY	Female	0	Yes	No	37	Yes	Yes	Fiber optic	
5021	6617-WLBQC	Female	0	Yes	Yes	23	Yes	Yes	Fiber optic	
3125	5222-JCXZT	Male	0	No	No	4	Yes	No	No	No int se

In [167]:

```
# Drop 'customerID' column since it is of no use:
df.drop('customerID',axis='columns',inplace=True)
```

In [168]:

```
df.sample(5)
```

Out[168]:

	gender	SeniorCitizen	Partner	Dependents	tenure	PhoneService	MultipleLines	InternetService	OnlineSecurity	Online
6907	Female	0	No	No	41	Yes	No	Fiber optic	No	
924	Female	1	No	No	22	Yes	No	Fiber optic	Yes	
5716	Male	0	No	Yes	2	Yes	No	DSL	No	

	gender	SeniorCitizen	Partner	Dependents	tenure	PhoneService	MultipleLines	InternetService	OnlineSecurity	Online
5136	Male	1	No	No	2	Yes	Yes	Fiber optic	No	
2179	**Female**	**0**	**No**	**No**	**5**	**Yes**	**No**	**No**	**No internet service**	**No**

In [169]:

```
# Data-types of all attributes:
df.dtypes
```

Out[169]:

```
gender                object
SeniorCitizen          int64
Partner               object
Dependents            object
tenure                 int64
PhoneService          object
MultipleLines         object
InternetService       object
OnlineSecurity        object
OnlineBackup          object
DeviceProtection      object
TechSupport           object
StreamingTV           object
StreamingMovies       object
Contract              object
PaperlessBilling      object
PaymentMethod         object
MonthlyCharges       float64
TotalCharges          object
Churn                 object
dtype: object
```

In [170]:

```
# Observe values in the "TotalCharges' column:
df["TotalCharges"].values
```

Out[170]:

```
array(['29.85', '1889.5', '108.15', ..., '346.45', '306.6', '6844.5'],
      shape=(7043,), dtype=object)
```

In [171]:

```
# Convert the above values into numeric-type:

# Step1) First check for any missing values:
df["TotalCharges"].isnull().sum()
```

Out[171]:

```
np.int64(0)
```

In [172]:

```
# Step2) Check for any blank-strings:
status=0
for i in df["TotalCharges"].values:
    if(i==" "):
        status = 1
        break
if(status == 1):
    print("Blank-strings are located")
else:
    print("No Blank strings are observed")
```

```
Blank-strings are located
```

In [173]:

```
# We have some blank values. Hence we will set 'errors' to 'coerce':

pd.to_numeric(df["TotalCharges"],errors='coerce').isnull()
```

Out[173]:

```
0       False
1       False
2       False
3       False
4       False
        ...
7038    False
7039    False
7040    False
7041    False
7042    False
Name: TotalCharges, Length: 7043, dtype: bool
```

In [174]:

```
df[pd.to_numeric(df.TotalCharges,errors='coerce').isnull()]
```

Out[174]:

	gender	SeniorCitizen	Partner	Dependents	tenure	PhoneService	MultipleLines	InternetService	OnlineSecurity	Online
488	Female	0	Yes	Yes	0	No	No phone service	DSL	Yes	
753	Male	0	No	Yes	0	Yes	No	No	No internet service	No
936	Female	0	Yes	Yes	0	Yes	No	DSL	Yes	
1082	Male	0	Yes	Yes	0	Yes	Yes	No	No internet service	No
1340	Female	0	Yes	Yes	0	No	No phone service	DSL	Yes	
3331	Male	0	Yes	Yes	0	Yes	No	No	No internet service	No
3826	Male	0	Yes	Yes	0	Yes	Yes	No	No internet service	No
4380	Female	0	Yes	Yes	0	Yes	No	No	No internet service	No
5218	Male	0	Yes	Yes	0	Yes	No	No	No internet service	No
6670	Female	0	Yes	Yes	0	Yes	Yes	DSL	No	
6754	Male	0	No	Yes	0	Yes	Yes	DSL	Yes	

In [175]:

```
#Remove rows with space in "TotalCharges" column:
df1 = df[df.TotalCharges!=' ']
```

In [176]:

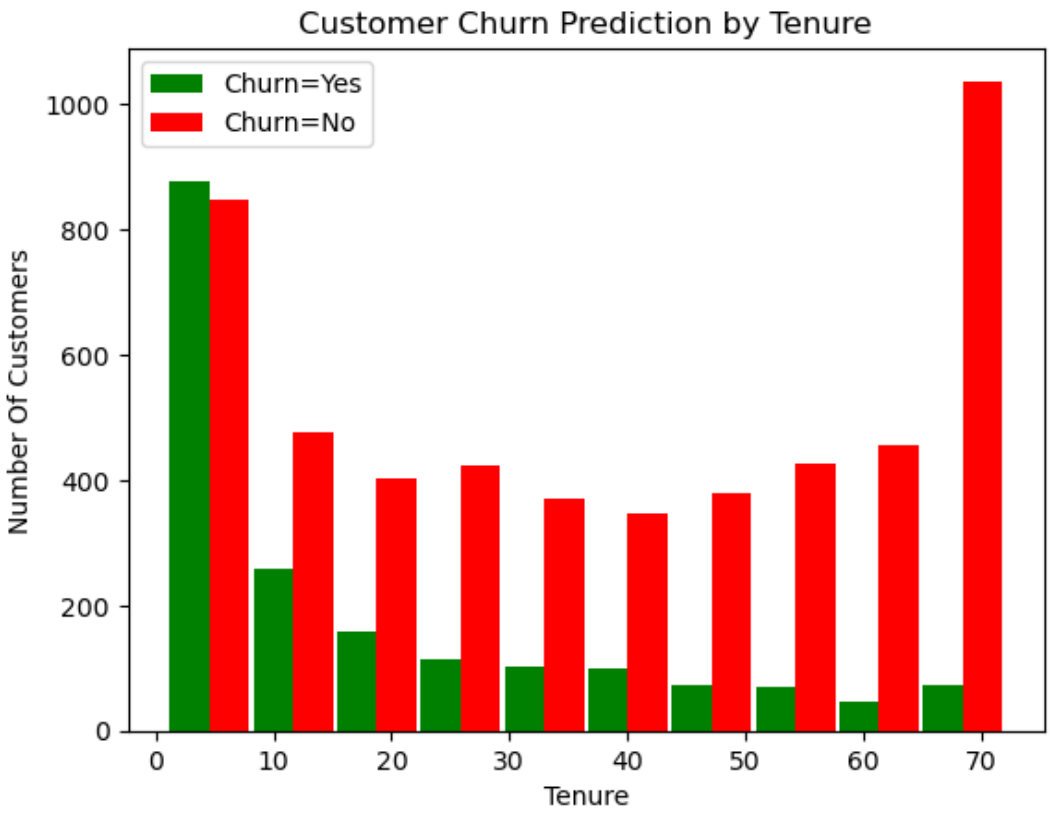
```
#Now convert the "TotalCharges" column to numeric:

df1.TotalCharges = pd.to_numeric(df1.TotalCharges)
```

In [177]:

```
#Customer Churn Prediction:
tenure_churn_no = df1[df1.Churn=='No'].tenure
tenure_churn_yes = df1[df1.Churn=='Yes'].tenure

plt.xlabel("Tenure")
plt.ylabel("Number Of Customers")
plt.title("Customer Churn Prediction by Tenure")

blood_sugar_men = [113, 85, 90, 150, 149, 88, 93, 115, 135, 80, 77, 82, 129]
blood_sugar_women = [67, 98, 89, 120, 133, 150, 84, 69, 89, 79, 120, 112, 100]

plt.hist([tenure_churn_yes, tenure_churn_no], rwidth=0.95, color=['green','red'],label=[
'Churn=Yes','Churn=No'])
plt.legend()
```

Out[177]:

```
<matplotlib.legend.Legend at 0x2420dc78190>
```

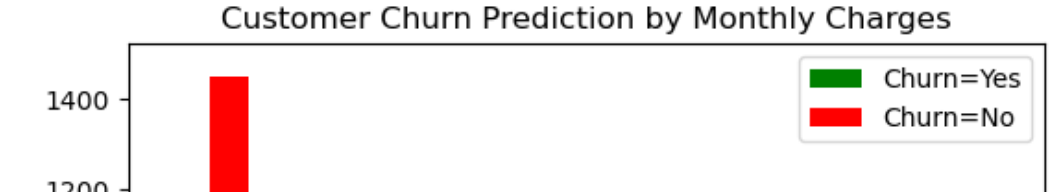


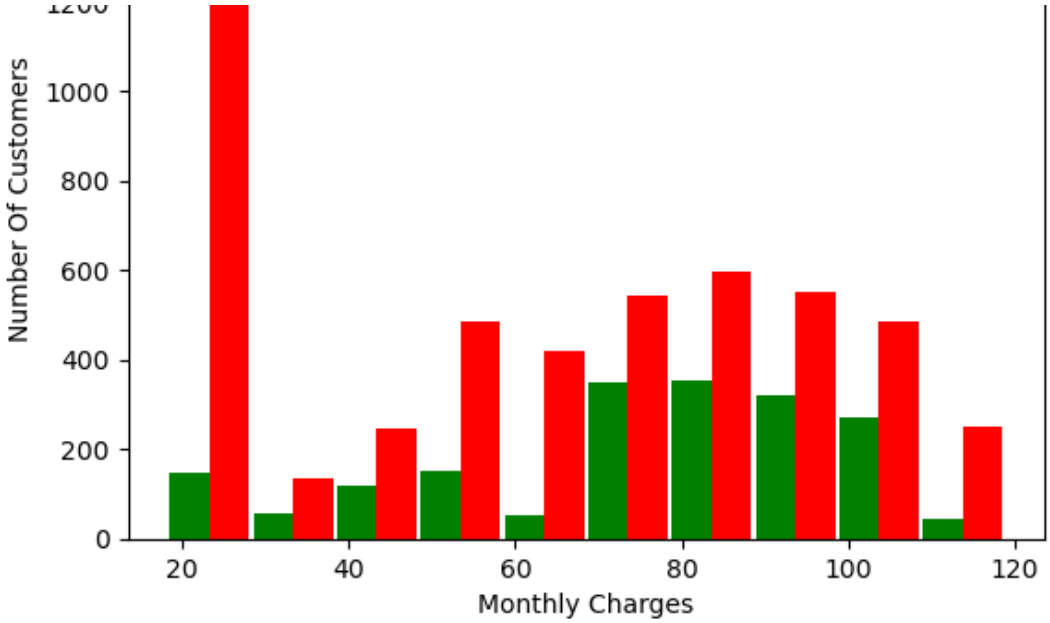
In [178]:

```
mc_churn_no = df1[df1.Churn=='No'].MonthlyCharges
mc_churn_yes = df1[df1.Churn=='Yes'].MonthlyCharges

plt.xlabel("Monthly Charges")
plt.ylabel("Number Of Customers")
plt.title("Customer Churn Prediction by Monthly Charges")

blood_sugar_men = [113, 85, 90, 150, 149, 88, 93, 115, 135, 80, 77, 82, 129]
blood_sugar_women = [67, 98, 89, 120, 133, 150, 84, 69, 89, 79, 120, 112, 100]

plt.hist([mc_churn_yes, mc_churn_no], rwidth=0.95, color=['green','red'],label=['Churn=Y
es','Churn=No'])
plt.legend()
```

Out[178]:

```
<matplotlib.legend.Legend at 0x2420e51d450>
```





In [179]:

```
# Now let us see unique values and their frequencies in all categorical attributes:

def print_unique_col_values(df):
       for column in df:
            if df[column].dtypes=='object':
                print("\n---------------------------\n\n")
                print(f'{column}: {df[column].unique()}')
                print(f'Frequency =  {len(df[column].unique())}')
                print("\n---------------------------\n\n")
print_unique_col_values(df1)
```

```
---------------------------


gender: ['Female' 'Male']
Frequency =  2

---------------------------



---------------------------


Partner: ['Yes' 'No']
Frequency =  2

---------------------------



---------------------------


Dependents: ['No' 'Yes']
Frequency =  2

---------------------------



---------------------------


PhoneService: ['No' 'Yes']
Frequency =  2

---------------------------
```

```
---------------------------


MultipleLines: ['No phone service' 'No' 'Yes']
Frequency =  3

---------------------------



---------------------------


InternetService: ['DSL' 'Fiber optic' 'No']
Frequency =  3

---------------------------



---------------------------


OnlineSecurity: ['No' 'Yes' 'No internet service']
Frequency =  3

---------------------------



---------------------------


OnlineBackup: ['Yes' 'No' 'No internet service']
Frequency =  3

---------------------------



---------------------------


DeviceProtection: ['No' 'Yes' 'No internet service']
Frequency =  3

---------------------------



---------------------------


TechSupport: ['No' 'Yes' 'No internet service']
Frequency =  3

---------------------------



---------------------------


StreamingTV: ['No' 'Yes' 'No internet service']
Frequency =  3

---------------------------
```

```
---------------------------


StreamingMovies: ['No' 'Yes' 'No internet service']
Frequency =  3

---------------------------



---------------------------


Contract: ['Month-to-month' 'One year' 'Two year']
Frequency =  3

---------------------------



---------------------------


PaperlessBilling: ['Yes' 'No']
Frequency =  2

---------------------------



---------------------------


PaymentMethod: ['Electronic check' 'Mailed check' 'Bank transfer (automatic)'
 'Credit card (automatic)']
Frequency =  4

---------------------------



---------------------------


Churn: ['No' 'Yes']
Frequency =  2

---------------------------
```

In [180]:

```
#Inspect the data:
df1.head()
```

Out[180]:

	gender	SeniorCitizen	Partner	Dependents	tenure	PhoneService	MultipleLines	InternetService	OnlineSecurity	OnlineBac
0	**Female**	**0**	**Yes**	**No**	**1**	**No**	**No phone service**	**DSL**	**No**	
1	**Male**	**0**	**No**	**No**	**34**	**Yes**	**No**	**DSL**	**Yes**	
2	**Male**	**0**	**No**	**No**	**2**	**Yes**	**No**	**DSL**	**Yes**	
3	**Male**	**0**	**No**	**No**	**45**	**No**	**No phone service**	**DSL**	**Yes**	

	gender	SeniorCitizen	Partner	Dependents	tenure	PhoneService	MultipleLines	InternetService	OnlineSecurity	OnlineBac
4	Female	0	No	No	2	Yes	No	Fiber optic	No	

In [181]:

```
# We observe that some columns show 'no internet service' or 'no phone service'. Let us r
eplace them with a simple No:
df1.replace('No internet service','No',inplace=True)
df1.replace('No phone service','No',inplace=True)
```

In [182]:

```
#Inspect the data once again:
df1.head()
```

Out[182]:

	gender	SeniorCitizen	Partner	Dependents	tenure	PhoneService	MultipleLines	InternetService	OnlineSecurity	OnlineBac
0	Female	0	Yes	No	1	No	No	DSL	No	
1	Male	0	No	No	34	Yes	No	DSL	Yes	
2	Male	0	No	No	2	Yes	No	DSL	Yes	
3	Male	0	No	No	45	No	No	DSL	Yes	
4	Female	0	No	No	2	Yes	No	Fiber optic	No	

In [183]:

```
# Let us now check again for all unique values in the categorical columns along with thei
r frequencies:
print_unique_col_values(df1)
```

```
---------------------------


gender: ['Female' 'Male']
Frequency =  2

---------------------------



---------------------------


Partner: ['Yes' 'No']
Frequency =  2

---------------------------



---------------------------


Dependents: ['No' 'Yes']
Frequency =  2

---------------------------
```

```
---------------------------


PhoneService: ['No' 'Yes']
Frequency =  2

---------------------------



---------------------------


MultipleLines: ['No' 'Yes']
Frequency =  2

---------------------------



---------------------------


InternetService: ['DSL' 'Fiber optic' 'No']
Frequency =  3

---------------------------



---------------------------


OnlineSecurity: ['No' 'Yes']
Frequency =  2

---------------------------



---------------------------


OnlineBackup: ['Yes' 'No']
Frequency =  2

---------------------------



---------------------------


DeviceProtection: ['No' 'Yes']
Frequency =  2

---------------------------



---------------------------


TechSupport: ['No' 'Yes']
Frequency =  2

---------------------------
```

```
---------------------------


StreamingTV: ['No' 'Yes']
Frequency =  2

---------------------------



---------------------------


StreamingMovies: ['No' 'Yes']
Frequency =  2

---------------------------



---------------------------


Contract: ['Month-to-month' 'One year' 'Two year']
Frequency =  3

---------------------------



---------------------------


PaperlessBilling: ['Yes' 'No']
Frequency =  2

---------------------------



---------------------------


PaymentMethod: ['Electronic check' 'Mailed check' 'Bank transfer (automatic)'
 'Credit card (automatic)']
Frequency =  4

---------------------------



---------------------------


Churn: ['No' 'Yes']
Frequency =  2

---------------------------
```

In [184]:

```
#Now we will convert all "Yes" and "No" values (in the above dataframe) to 1 and 0 respec
tively so that our neural network model will understand it clearly:
yes_no_columns = ['Partner','Dependents','PhoneService','MultipleLines','OnlineSecurity',
'OnlineBackup',
                  'DeviceProtection','TechSupport','StreamingTV','StreamingMovies','Pape
rlessBilling','Churn']
for col in yes_no_columns:
```

```
    df1[col].replace({'Yes': 1,'No': 0},inplace=True)
```

In [185]:

```
#Run the User-defined function print_unique_col_values() once again:
# Run this BEFORE df1.head()
for col in df1:
    print(f'{col}: {df1[col].unique()}')
#print_unique_col_values(df1)
```

```
gender: ['Female' 'Male']
SeniorCitizen: [0 1]
Partner: [1 0]
Dependents: [0 1]
tenure: [ 1 34  2 45  8 22 10 28 62 13 16 58 49 25 69 52 71 21 12 30 47 72 17 27
  5 46 11 70 63 43 15 60 18 66  9  3 31 50 64 56  7 42 35 48 29 65 38 68
 32 55 37 36 41  6  4 33 67 23 57 61 14 20 53 40 59 24 44 19 54 51 26 39]
PhoneService: [0 1]
MultipleLines: [0 1]
InternetService: ['DSL' 'Fiber optic' 'No']
OnlineSecurity: [0 1]
OnlineBackup: [1 0]
DeviceProtection: [0 1]
TechSupport: [0 1]
StreamingTV: [0 1]
StreamingMovies: [0 1]
Contract: ['Month-to-month' 'One year' 'Two year']
PaperlessBilling: [1 0]
PaymentMethod: ['Electronic check' 'Mailed check' 'Bank transfer (automatic)'
 'Credit card (automatic)']
MonthlyCharges: [29.85 56.95 53.85 ... 63.1  44.2  78.7 ]
TotalCharges: [  29.85 1889.5   108.15 ...  346.45  306.6  6844.5 ]
Churn: [0 1]
```

In [186]:

```
# Similary change Female into 1 and Male into 0 respectively:
df1['gender'].replace({'Female':1,'Male':0},inplace=True)
df1.gender.unique()
```

Out[186]:

```
array([1, 0])
```

In [187]:

```
# Apply one-hot encoding to categorical columns:
df2 = pd.get_dummies(data=df1, columns=['InternetService','Contract','PaymentMethod'])
df2.columns
```

Out[187]:

```
Index(['gender', 'SeniorCitizen', 'Partner', 'Dependents', 'tenure',
       'PhoneService', 'MultipleLines', 'OnlineSecurity', 'OnlineBackup',
       'DeviceProtection', 'TechSupport', 'StreamingTV', 'StreamingMovies',
       'PaperlessBilling', 'MonthlyCharges', 'TotalCharges', 'Churn',
       'InternetService_DSL', 'InternetService_Fiber optic',
       'InternetService_No', 'Contract_Month-to-month', 'Contract_One year',
       'Contract_Two year', 'PaymentMethod_Bank transfer (automatic)',
       'PaymentMethod_Credit card (automatic)',
       'PaymentMethod_Electronic check', 'PaymentMethod_Mailed check'],
      dtype='object')
```

In [188]:

```
df2.dtypes
```

Out[188]:

```
gender                                        int64
SeniorCitizen                                 int64
Partner                                       int64
Dependents                                    int64
```

```
tenure                                      int64
PhoneService                                int64
MultipleLines                               int64
OnlineSecurity                              int64
OnlineBackup                                int64
DeviceProtection                            int64
TechSupport                                 int64
StreamingTV                                 int64
StreamingMovies                             int64
PaperlessBilling                            int64
MonthlyCharges                            float64
TotalCharges                              float64
Churn                                       int64
InternetService_DSL                          bool
InternetService_Fiber optic                  bool
InternetService_No                           bool
Contract_Month-to-month                      bool
Contract_One year                            bool
Contract_Two year                            bool
PaymentMethod_Bank transfer (automatic)      bool
PaymentMethod_Credit card (automatic)        bool
PaymentMethod_Electronic check               bool
PaymentMethod_Mailed check                   bool
dtype: object
```

In [189]:

```python
# Apply MinMaxScaling to "tenure","MonthlyCharges" and "TotalCharges" columns:
cols_to_scale = ['tenure','MonthlyCharges','TotalCharges']

from sklearn.preprocessing import MinMaxScaler
scaler = MinMaxScaler()
df2[cols_to_scale] = scaler.fit_transform(df2[cols_to_scale])
```

In [190]:

```python
for col in df2:
    print(f'{col}: {df2[col].unique()}')
```

```
gender: [1 0]
SeniorCitizen: [0 1]
Partner: [1 0]
Dependents: [0 1]
tenure: [0.         0.46478873 0.01408451 0.61971831 0.09859155 0.29577465
 0.12676056 0.38028169 0.85915493 0.16901408 0.21126761 0.8028169
 0.67605634 0.33802817 0.95774648 0.71830986 0.98591549 0.28169014
 0.15492958 0.4084507  0.64788732 1.         0.22535211 0.36619718
 0.05633803 0.63380282 0.14084507 0.97183099 0.87323944 0.5915493
 0.1971831  0.83098592 0.23943662 0.91549296 0.11267606 0.02816901
 0.42253521 0.69014085 0.88732394 0.77464789 0.08450704 0.57746479
 0.47887324 0.66197183 0.3943662  0.90140845 0.52112676 0.94366197
 0.43661972 0.76056338 0.50704225 0.49295775 0.56338028 0.07042254
 0.04225352 0.45070423 0.92957746 0.30985915 0.78873239 0.84507042
 0.18309859 0.26760563 0.73239437 0.54929577 0.81690141 0.32394366
 0.6056338  0.25352113 0.74647887 0.70422535 0.35211268 0.53521127]
PhoneService: [0 1]
MultipleLines: [0 1]
OnlineSecurity: [0 1]
OnlineBackup: [1 0]
DeviceProtection: [0 1]
TechSupport: [0 1]
StreamingTV: [0 1]
StreamingMovies: [0 1]
PaperlessBilling: [1 0]
MonthlyCharges: [0.11542289 0.38507463 0.35422886 ... 0.44626866 0.25820896 0.60149254]
TotalCharges: [0.0012751  0.21586661 0.01031041 ... 0.03780868 0.03321025 0.78764136]
Churn: [0 1]
InternetService_DSL: [ True False]
InternetService_Fiber optic: [False  True]
InternetService_No: [False  True]
Contract_Month-to-month: [ True False]
Contract_One year: [False  True]
```

```
Contract_Two year: [False  True]
PaymentMethod_Bank transfer (automatic): [False  True]
PaymentMethod_Credit card (automatic): [False  True]
PaymentMethod_Electronic check: [ True False]
PaymentMethod_Mailed check: [False  True]
```

In [ ]:

In [ ]:

In [191]:

```
#Apply train-test split:
X = df2.drop('Churn',axis='columns')
y = df2['Churn']

from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(X,y,test_size=0.2,random_state=5)
```

In [192]:

```
#Build a ANN model around this:
import tensorflow as tf
from tensorflow import keras


model = keras.Sequential([
    keras.layers.Dense(26, input_shape=(26,), activation='relu'),
    keras.layers.Dense(21, activation='relu'),
    keras.layers.Dense(1, activation='sigmoid')
])

# opt = keras.optimizers.Adam(learning_rate=0.01)

model.compile(optimizer='adam',
              loss='binary_crossentropy',
              metrics=['accuracy'])

model.fit(X_train, y_train, epochs=100)
```

```
Epoch 1/100
176/176 ━━━━━━━━━━━━━━━━━━━━ 2s 2ms/step - accuracy: 0.7607 - loss: 0.4915
Epoch 2/100
176/176 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.7938 - loss: 0.4257
Epoch 3/100
176/176 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step - accuracy: 0.8009 - loss: 0.4189
Epoch 4/100
176/176 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step - accuracy: 0.8018 - loss: 0.4153
Epoch 5/100
176/176 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step - accuracy: 0.8057 - loss: 0.4123
Epoch 6/100
176/176 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.8060 - loss: 0.4096
Epoch 7/100
176/176 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step - accuracy: 0.8055 - loss: 0.4080
Epoch 8/100
176/176 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step - accuracy: 0.8091 - loss: 0.4059
Epoch 9/100
176/176 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.8073 - loss: 0.4045
Epoch 10/100
176/176 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - accuracy: 0.8135 - loss: 0.4040
Epoch 11/100
176/176 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.8123 - loss: 0.4011
Epoch 12/100
176/176 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step - accuracy: 0.8148 - loss: 0.4001
Epoch 13/100
176/176 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step - accuracy: 0.8155 - loss: 0.3991
Epoch 14/100
```

```
176/176 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step - accuracy: 0.8149 - loss: 0.3983
Epoch 15/100
176/176 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.8153 - loss: 0.3961
Epoch 16/100
176/176 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step - accuracy: 0.8126 - loss: 0.3958
Epoch 17/100
176/176 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - accuracy: 0.8165 - loss: 0.3940
Epoch 18/100
176/176 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step - accuracy: 0.8123 - loss: 0.3931
Epoch 19/100
176/176 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - accuracy: 0.8158 - loss: 0.3920
Epoch 20/100
176/176 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - accuracy: 0.8164 - loss: 0.3915
Epoch 21/100
176/176 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step - accuracy: 0.8220 - loss: 0.3894
Epoch 22/100
176/176 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - accuracy: 0.8187 - loss: 0.3890
Epoch 23/100
176/176 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - accuracy: 0.8215 - loss: 0.3881
Epoch 24/100
176/176 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.8192 - loss: 0.3883
Epoch 25/100
176/176 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step - accuracy: 0.8212 - loss: 0.3854
Epoch 26/100
176/176 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step - accuracy: 0.8188 - loss: 0.3849
Epoch 27/100
176/176 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step - accuracy: 0.8196 - loss: 0.3849
Epoch 28/100
176/176 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step - accuracy: 0.8176 - loss: 0.3833
Epoch 29/100
176/176 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - accuracy: 0.8199 - loss: 0.3829
Epoch 30/100
176/176 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step - accuracy: 0.8219 - loss: 0.3816
Epoch 31/100
176/176 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step - accuracy: 0.8215 - loss: 0.3808
Epoch 32/100
176/176 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.8240 - loss: 0.3799
Epoch 33/100
176/176 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step - accuracy: 0.8220 - loss: 0.3784
Epoch 34/100
176/176 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step - accuracy: 0.8240 - loss: 0.3775
Epoch 35/100
176/176 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.8268 - loss: 0.3768
Epoch 36/100
176/176 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - accuracy: 0.8224 - loss: 0.3766
Epoch 37/100
176/176 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - accuracy: 0.8261 - loss: 0.3763
Epoch 38/100
176/176 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.8219 - loss: 0.3759
Epoch 39/100
176/176 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - accuracy: 0.8238 - loss: 0.3750
Epoch 40/100
176/176 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - accuracy: 0.8258 - loss: 0.3734
Epoch 41/100
176/176 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.8265 - loss: 0.3738
Epoch 42/100
176/176 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.8240 - loss: 0.3714
Epoch 43/100
176/176 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - accuracy: 0.8236 - loss: 0.3706
Epoch 44/100
176/176 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step - accuracy: 0.8267 - loss: 0.3710
Epoch 45/100
176/176 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.8267 - loss: 0.3691
Epoch 46/100
176/176 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step - accuracy: 0.8284 - loss: 0.3695
Epoch 47/100
176/176 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.8258 - loss: 0.3689
Epoch 48/100
176/176 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.8288 - loss: 0.3676
Epoch 49/100
176/176 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - accuracy: 0.8302 - loss: 0.3669
Epoch 50/100
```

```
176/176 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.8315 - loss: 0.3655
Epoch 51/100
176/176 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - accuracy: 0.8304 - loss: 0.3658
Epoch 52/100
176/176 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.8300 - loss: 0.3649
Epoch 53/100
176/176 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step - accuracy: 0.8279 - loss: 0.3635
Epoch 54/100
176/176 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step - accuracy: 0.8322 - loss: 0.3629
Epoch 55/100
176/176 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step - accuracy: 0.8288 - loss: 0.3633
Epoch 56/100
176/176 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step - accuracy: 0.8313 - loss: 0.3622
Epoch 57/100
176/176 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.8343 - loss: 0.3607
Epoch 58/100
176/176 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.8277 - loss: 0.3607
Epoch 59/100
176/176 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step - accuracy: 0.8308 - loss: 0.3597
Epoch 60/100
176/176 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.8295 - loss: 0.3601
Epoch 61/100
176/176 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - accuracy: 0.8295 - loss: 0.3601
Epoch 62/100
176/176 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.8316 - loss: 0.3597
Epoch 63/100
176/176 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step - accuracy: 0.8357 - loss: 0.3582
Epoch 64/100
176/176 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step - accuracy: 0.8308 - loss: 0.3584
Epoch 65/100
176/176 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.8331 - loss: 0.3565
Epoch 66/100
176/176 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step - accuracy: 0.8309 - loss: 0.3564
Epoch 67/100
176/176 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step - accuracy: 0.8320 - loss: 0.3563
Epoch 68/100
176/176 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - accuracy: 0.8327 - loss: 0.3551
Epoch 69/100
176/176 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step - accuracy: 0.8350 - loss: 0.3541
Epoch 70/100
176/176 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step - accuracy: 0.8366 - loss: 0.3534
Epoch 71/100
176/176 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step - accuracy: 0.8370 - loss: 0.3527
Epoch 72/100
176/176 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step - accuracy: 0.8356 - loss: 0.3521
Epoch 73/100
176/176 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step - accuracy: 0.8343 - loss: 0.3522
Epoch 74/100
176/176 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.8357 - loss: 0.3507
Epoch 75/100
176/176 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - accuracy: 0.8370 - loss: 0.3507
Epoch 76/100
176/176 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step - accuracy: 0.8357 - loss: 0.3506
Epoch 77/100
176/176 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step - accuracy: 0.8382 - loss: 0.3495
Epoch 78/100
176/176 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.8375 - loss: 0.3496
Epoch 79/100
176/176 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.8391 - loss: 0.3483
Epoch 80/100
176/176 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step - accuracy: 0.8373 - loss: 0.3490
Epoch 81/100
176/176 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step - accuracy: 0.8380 - loss: 0.3485
Epoch 82/100
176/176 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step - accuracy: 0.8366 - loss: 0.3480
Epoch 83/100
176/176 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step - accuracy: 0.8389 - loss: 0.3467
Epoch 84/100
176/176 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step - accuracy: 0.8388 - loss: 0.3465
Epoch 85/100
176/176 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step - accuracy: 0.8395 - loss: 0.3452
Epoch 86/100
```

```
176/176 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.8407 - loss: 0.3447
Epoch 87/100
176/176 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - accuracy: 0.8356 - loss: 0.3454
Epoch 88/100
176/176 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - accuracy: 0.8375 - loss: 0.3452
Epoch 89/100
176/176 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step - accuracy: 0.8409 - loss: 0.3434
Epoch 90/100
176/176 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - accuracy: 0.8423 - loss: 0.3422
Epoch 91/100
176/176 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - accuracy: 0.8418 - loss: 0.3422
Epoch 92/100
176/176 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.8405 - loss: 0.3415
Epoch 93/100
176/176 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.8379 - loss: 0.3418
Epoch 94/100
176/176 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.8398 - loss: 0.3416
Epoch 95/100
176/176 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step - accuracy: 0.8411 - loss: 0.3403
Epoch 96/100
176/176 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step - accuracy: 0.8411 - loss: 0.3412
Epoch 97/100
176/176 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - accuracy: 0.8393 - loss: 0.3400
Epoch 98/100
176/176 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step - accuracy: 0.8409 - loss: 0.3391
Epoch 99/100
176/176 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.8421 - loss: 0.3377
Epoch 100/100
176/176 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step - accuracy: 0.8448 - loss: 0.3386
```

Out[192]:

```
<keras.src.callbacks.history.History at 0x2420e576850>
```

In [193]:

```
#Evaluate the ANN model on test-dataset:
model.evaluate(X_test, y_test)
```

```
44/44 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step - accuracy: 0.7811 - loss: 0.4835
```

Out[193]:

```
[0.48349785804748535, 0.7810945510864258]
```

In [194]:

```
#Now make predictions on input type test data:
yp = model.predict(X_test)

y_pred = []
for element in yp:
    if element > 0.5:
        y_pred.append(1)
    else:
        y_pred.append(0)
```

```
44/44 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step
```

In [195]:

```
#Plot confusion matrix:
from sklearn.metrics import confusion_matrix , classification_report

print(classification_report(y_test,y_pred))
```

```
              precision    recall  f1-score   support

           0       0.85      0.84      0.85       999
           1       0.62      0.62      0.62       408

    accuracy                           0.78      1407
   macro avg       0.73      0.73      0.73      1407
```

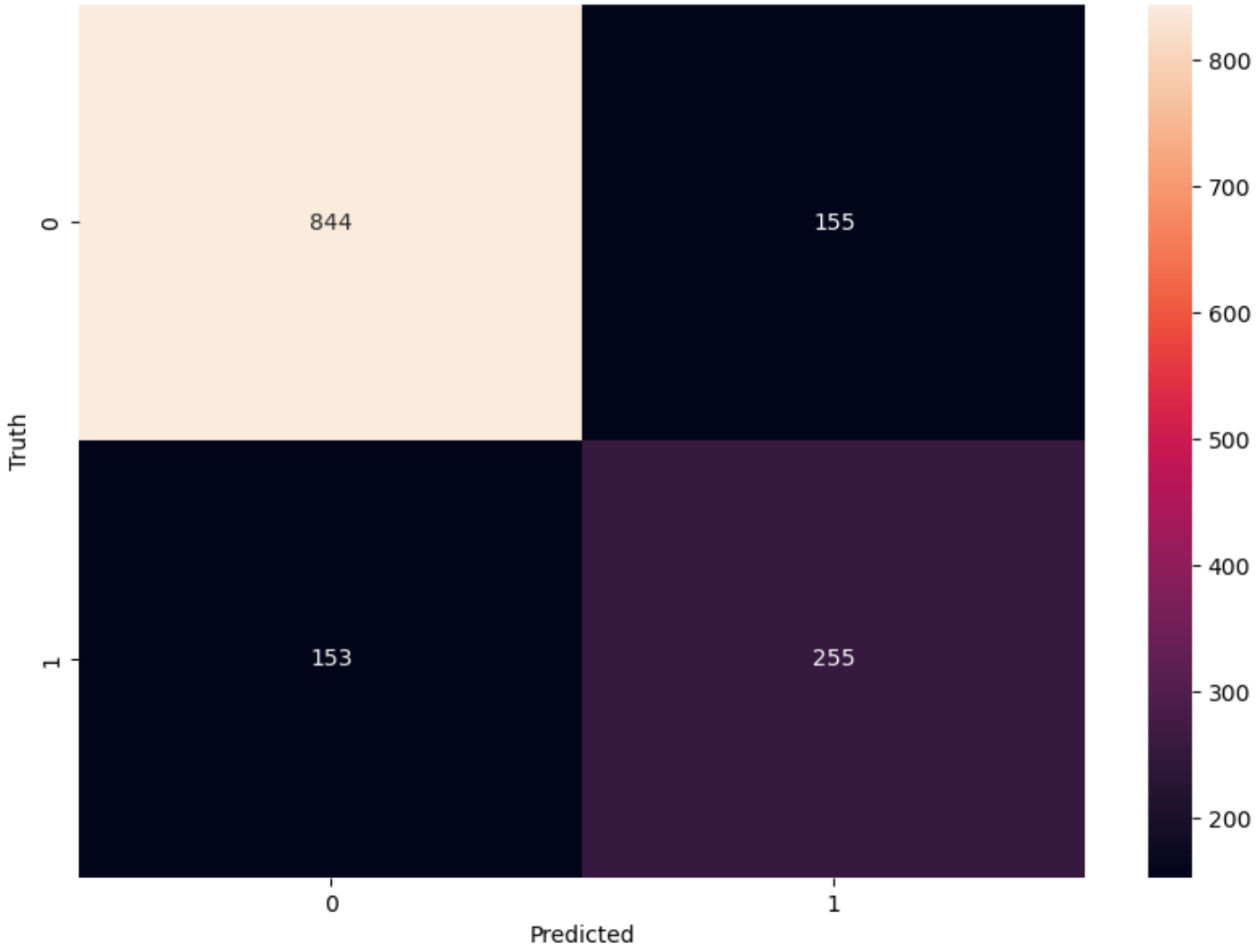
```
weighted avg       0.78      0.78      0.78      1407
```

In [196]:

```
import seaborn as sn
cm = tf.math.confusion_matrix(labels=y_test,predictions=y_pred)

plt.figure(figsize = (10,7))
sn.heatmap(cm, annot=True, fmt='d')
plt.xlabel('Predicted')
plt.ylabel('Truth')
```

Out[196]:

```
Text(95.72222222222221, 0.5, 'Truth')
```



In [ ]:

In [ ]:

In [ ]: